HOUSE RENT

Broker-Mediated Real Estate Platform

Full Stack Development with MERN — Project Documentation

Prepared By

PRAVEEN KUMAR PENUGONDA(TEAM LEAD)

RAGA PRIYA ISWARYA YASALAPU(TEAM MEMBER)

July 2026

Table of Contents

1. Introduction

1.1 Project Title

House Rent — a broker-mediated real estate rental and sale platform.

1.2 Team Members- And ROLES

- Raga Priya Iswarya Yasalapu-TEAM MEMBER(FRONT END DEVELOPMENT,BACKEND DEVELOPMENT,DOCUMENTATION)
- Praveen Kumar Penugonda-TEAM LEAD (REPORTING)

2. Project Overview

2.1 Purpose

House Rent is a real estate marketplace built to solve a trust problem that plagues informal listings platforms: anyone can post a fake or duplicate ad, and there is no verification step before a renter or buyer hands over their contact details to a stranger. House Rent introduces a broker/admin role into that workflow. Property Owners submit listings for Rent or Sale; an Admin/Broker reviews and explicitly approves or rejects each one; and only approved, available listings ever reach Renters and Buyers, who can then request a booking or purchase with confidence.

2.2 Key Features

- Multi-role accounts — a single login can hold both an Owner and a Renter role. A user can add a second role to an existing account using the same email and password; if the account has more than one role, login prompts the user to choose which one to act as.
- Admin accounts cannot be self-registered — this is enforced in the UI and, independently, in the API itself. Admin accounts are created only through a server-side seed script.
- Listing approval workflow — every new property starts in “pending” status. Only properties that are both “approved” and marked available appear on the public browse page. Editing an already-approved listing returns it to “pending” for re-review.
- Rent and Sale support — listings can be marked for rent or for sale, with rent/sale-aware labelling throughout the UI.
- Search and filtering — renters and buyers can filter listings by location, property type, ad type (rent/sale), and price range.
- Owner Dashboard — Add Property (with image upload), All Properties (status and availability management), and All Bookings.
- Admin Dashboard — All Users (grant/revoke trusted-owner status, delete accounts), All Properties (approve/reject with a status filter), and All Bookings (a full audit view).
- Renter Dashboard — browse approved listings, request a booking or purchase, and track requests under Booking History.
- A dark, animated interface built with Tailwind CSS, JWT-based authentication, and image uploads handled through Multer.

3. Architecture

House Rent follows a conventional three-tier MERN architecture, with one deliberate addition: an approval gate enforced in the API layer that sits between a property being created and it becoming publicly visible.

3.1 Frontend — React 18 + Vite

- src/pages/ — Home, Login, Register, Properties, PropertyDetails, OwnerDashboard, AdminDashboard, RenterDashboard
- src/components/ — Navbar, PropertyCard, ProtectedRoute (role-gated routing), AnimatedBackground
- src/context/AuthContext.jsx — holds the current user and token, exposes login / register / logout
- src/api/axios.js — a shared Axios instance that automatically attaches the JWT from localStorage to every request
- Styling — Tailwind CSS

3.2 Backend — Node.js + Express

- routes/ — auth.js, properties.js, bookings.js, users.js
- middleware/auth.js — JWT verification (requireAuth) and role-based access control (requireRole)
- models/ — User, Property, Booking (Mongoose schemas)
- seed.js — creates admin accounts directly in the database, bypassing the public registration API entirely
- File uploads are handled via Multer and served statically from /uploads

3.3 Database — MongoDB via Mongoose

The application persists data across three core collections:

Collection	Key Fields	Notes
Users	name, email (unique), password (bcrypt-hashed), roles [ ], granted	roles is an array, so one account can hold both “owner” and “renter”; granted marks a trusted, admin-approved owner
Properties	ownerId (ref User), propertyType, adType, address, location, images[ ], ownerContact, amount, details, available, status, rejectionReason	status drives the approval workflow: pending → approved / rejected
Bookings	ownerId, propertyId, tenantId (all refs), tenantName, tenantPhone, message, status	status tracks each booking/enquiry through pending → booked / cancelled

4. Setup Instructions

4.1 Prerequisites

- Node.js (v18 or later recommended)
- MongoDB — a local instance or a MongoDB Atlas cluster
- Git

4.2 Installation

Clone the repository, then install dependencies for both the server and the client:

```
cd server
npm install

cd ../client
npm install
```

Create a .env file inside the server/ directory with the following variables:

```
MONGO_URI=<your MongoDB connection string>
JWT_SECRET=<a long, random secret string>
PORT=8000
```

5. Folder Structure

5.1 Client

```
client/
├── src/
│   ├── pages/        (Home, Login, Register, Properties, PropertyDetails,
│   │                 OwnerDashboard, AdminDashboard, RenterDashboard)
│   ├── components/   (Navbar, PropertyCard, ProtectedRoute, AnimatedBackground)
│   ├── context/      (AuthContext.jsx)
│   └── api/          (axios.js)
├── index.html
├── vite.config.js
└── tailwind.config.js
```

5.2 Server

```
server/
├── models/       (User.js, Property.js, Booking.js)
├── routes/       (auth.js, properties.js, bookings.js, users.js)
├── middleware/   (auth.js)
├── uploads/      (uploaded property images, served statically)
├── index.js      (application entry point)
└── seed.js       (creates admin accounts directly in the database)
```

6. Running the Application

Both the client and the server are run in development mode using npm run dev / nodemon, from two separate terminals.

6.1 Backend

```
cd server
npx nodemon index.js
```

The API starts on http://localhost:8000 and connects to MongoDB.

6.2 Frontend

```
cd client
npm run dev
```

The application is served on http://localhost:5173.

7. API Documentation

All endpoints are prefixed with /api. Authentication is via a Bearer JWT in the Authorization header, except where marked “—”.

7.1 Authentication Endpoints

Method	Route	Auth	Description
POST	/auth/register	—	Registers a Renter or Owner account. If the email already exists and the password matches, the requested role is added to that existing account instead of creating a duplicate.
POST	/auth/login	—	Authenticates a user. Returns needsRoleSelection: true if the account holds more than one role.
GET	/auth/me	JWT	Returns the currently authenticated user's profile.

7.2 Property Endpoints

Method	Route	Auth	Description
GET	/properties	—	Public listing, filterable by search, adType, propertyType, minPrice, maxPrice.
GET	/properties/mine	Owner	Returns the logged-in owner's own properties, in any status.
GET	/properties/admin/all	Admin	Returns all properties, with an optional ?status= filter.
GET	/properties/:id	—	Returns a single property's details.
POST	/properties	Owner	Creates a new property (multipart/form-data, images field for uploads).
PUT	/properties/:id	Owner / Admin	Updates an existing property.
PATCH	/properties/:id/status	Admin	Approves or rejects a pending property.
PATCH	/properties/:id/availability	Owner / Admin	Toggles a property's availability.
DELETE	/properties/:id	Owner / Admin	Deletes a property.

7.3 Booking Endpoints

Method	Route	Auth	Description
POST	/bookings	Renter	Creates a booking (rent) or enquiry (sale) request.
GET	/bookings/mine	Renter	Returns the logged-in renter's own bookings.
GET	/bookings/owner	Owner	Returns bookings received on the owner's properties.
GET	/bookings	Admin	Returns all bookings across the platform.
PATCH	/bookings/:id/status	Owner / Admin	Updates a booking's status.

7.4 User & Admin Endpoints

Method	Route	Auth	Description
GET	/users	Admin	Returns all registered users.
PATCH	/users/:id/grant	Admin	Grants or revokes an owner's trusted status.
DELETE	/users/:id	Admin	Deletes a user account.

8. Authentication

Authentication is JSON Web Token (JWT) based. On successful login, the server issues a signed token containing the user's ID, name, email, and roles, which the client stores and attaches to every subsequent request as an Authorization: Bearer <token> header. Tokens expire after seven days. Passwords are never stored in plain text — they are hashed with bcryptjs before being persisted.

Two middleware layers enforce access control on the server:

- requireAuth — verifies that a valid, unexpired token is present before allowing a request through.

- requireRole(...) — checks the authenticated user's roles against the roles permitted for that route, independently of anything the client sends.

For accounts holding more than one role, the login response indicates that a role choice is needed; the selected role for that session determines which dashboard and permissions apply.

9. User Interface

The interface uses a dark, glass-morphic theme built with Tailwind CSS, with a softly animated gradient background used consistently across every page to give the application a distinct, polished identity rather than a default framework look.

- Role-specific dashboards — Owner, Renter, and Admin each see only the navigation and actions relevant to their role, rather than a single generic dashboard.
- Consistent property cards — the same card design is reused across the landing page, the public browse page, and every dashboard, so listings look and behave the same everywhere.
- Status-aware UI — a property's pending / approved / rejected state and a booking's pending / booked state are always visible with clear colour coding, so users never have to guess where a request stands.
- Responsive layout — pages reflow for smaller viewports using Tailwind's responsive utilities.

10. Testing

10.1 Testing Strategy

The House Rent application was tested using manual functional testing to verify that all major modules work correctly, both individually and in combination with one another across the frontend, backend, and MongoDB database.

10.2 Testing Environment

- Frontend: React.js
- Backend: Node.js with Express.js
- Database: MongoDB Atlas
- Browser: Google Chrome
- API Testing: Postman and direct browser requests
- Development Tools: Visual Studio Code, MongoDB Compass

10.3 Test Cases

Module	Test Case	Expected Result	Status
Registration	Register new Admin, Owner, and Renter accounts	User registered successfully	Passed
Registration	Register using an existing email	“User already exists” message displayed	Passed
Login	Log in with valid credentials	Redirect to the respective dashboard	Passed
Login	Log in with an invalid password	Invalid password message displayed	Passed
Authentication	Access a protected route without logging in	Unauthorized access prevented	Passed
Owner Approval	Admin grants Owner access	Owner status updated successfully	Passed

Module	Test Case	Expected Result	Status
Property Management	Owner adds a property	Property stored successfully in MongoDB	Passed
Property Management	Owner updates property details	Updated property details displayed	Passed
Property Management	Owner deletes a property	Property removed successfully	Passed
Property Listing	Renter views available properties	Properties displayed correctly	Passed
Booking	Renter books a property	Booking created successfully	Passed
Booking Management	Owner updates booking status	Booking status updated	Passed
Admin Dashboard	View all users	All registered users displayed	Passed
Admin Dashboard	View all properties	All properties displayed	Passed
Admin Dashboard	View all bookings	All booking records displayed	Passed
Logout	User logs out	Session terminated and redirected to login	Passed

10.4 Functional Testing

The following modules were successfully tested:

- User registration and login
- JWT authentication and role-based authorization
- Admin, Owner, and Renter dashboards
- Property management (create, read, update, delete)
- Property listing and search
- Booking management
- MongoDB database operations
- Protected routes and session management

10.5 Integration Testing

Integration testing verified communication between:

- React frontend ↔ Express backend
- Express backend ↔ MongoDB Atlas
- Authentication middleware ↔ protected API routes
- Property module ↔ Booking module
- Admin module ↔ User module

All modules interacted successfully, with results matching expectations.

10.6 Database Testing

The MongoDB database was tested for:

- User registration and login authentication
- Property and booking record creation
- Owner approval status updates
- Data retrieval, update, and deletion

All CRUD operations completed successfully.

10.7 Browser Testing

The application was tested on Google Chrome and Microsoft Edge; the interface and functionality worked correctly on both.

10.8 Result

The House Rent application successfully passed manual functional testing across all major modules. Core functionality — authentication, role-based access, property management, and booking management — worked as expected throughout testing.

11. Screenshots / Demo

Demo Link: https://drive.google.com/file/d/1rW67_Q6fhhDazjiIrcXiGNcpeQqSqNBC/view?usp=sharing

GitHub Repository: https://github.com/ish-5/HouseRent

11.1 Landing Page

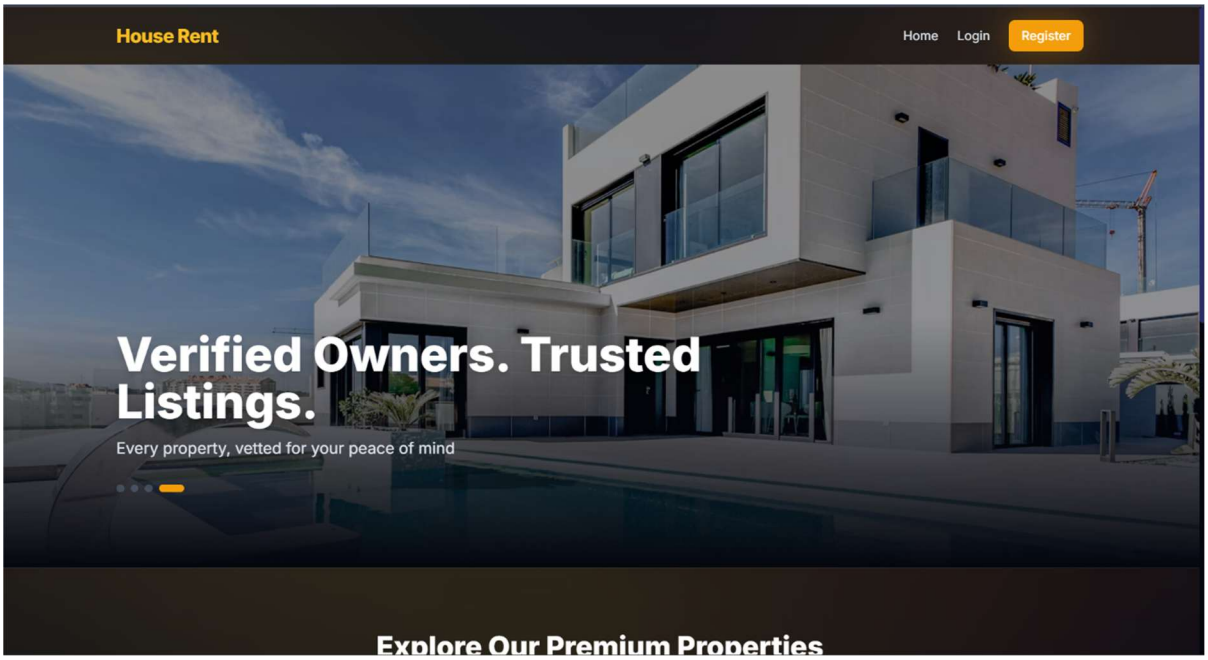

Figure 1: Landing page — hero section

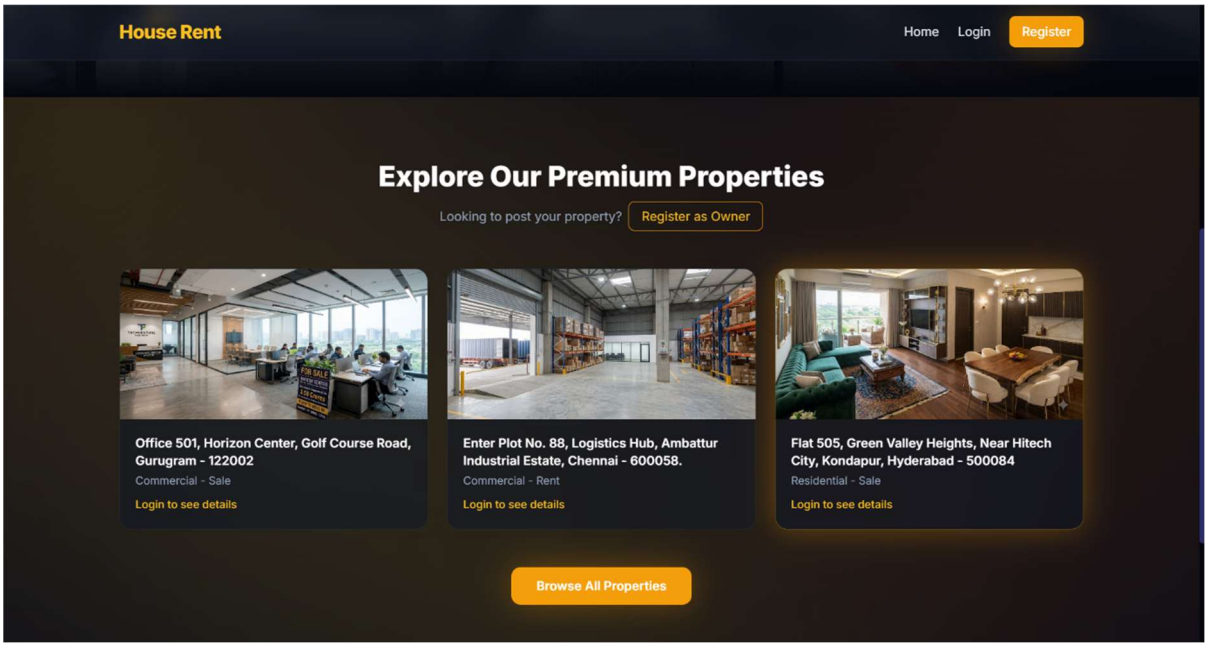

Figure 2: Landing page — featured properties

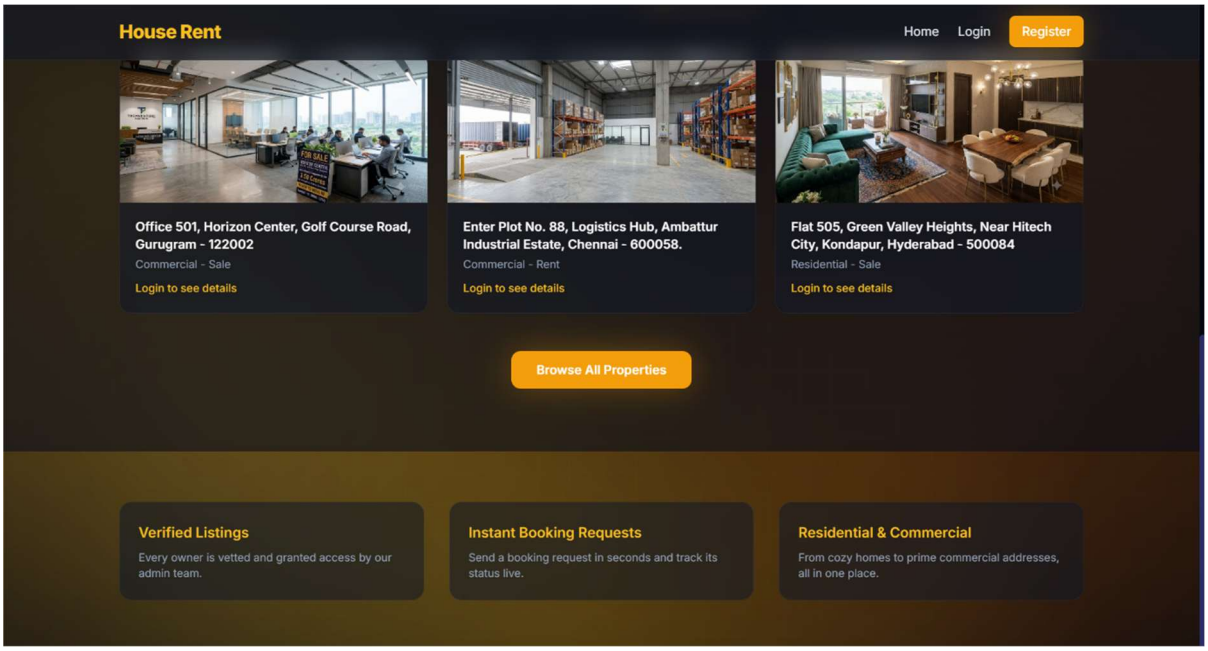

Figure 3: Landing page — property listings

11.2 Authentication

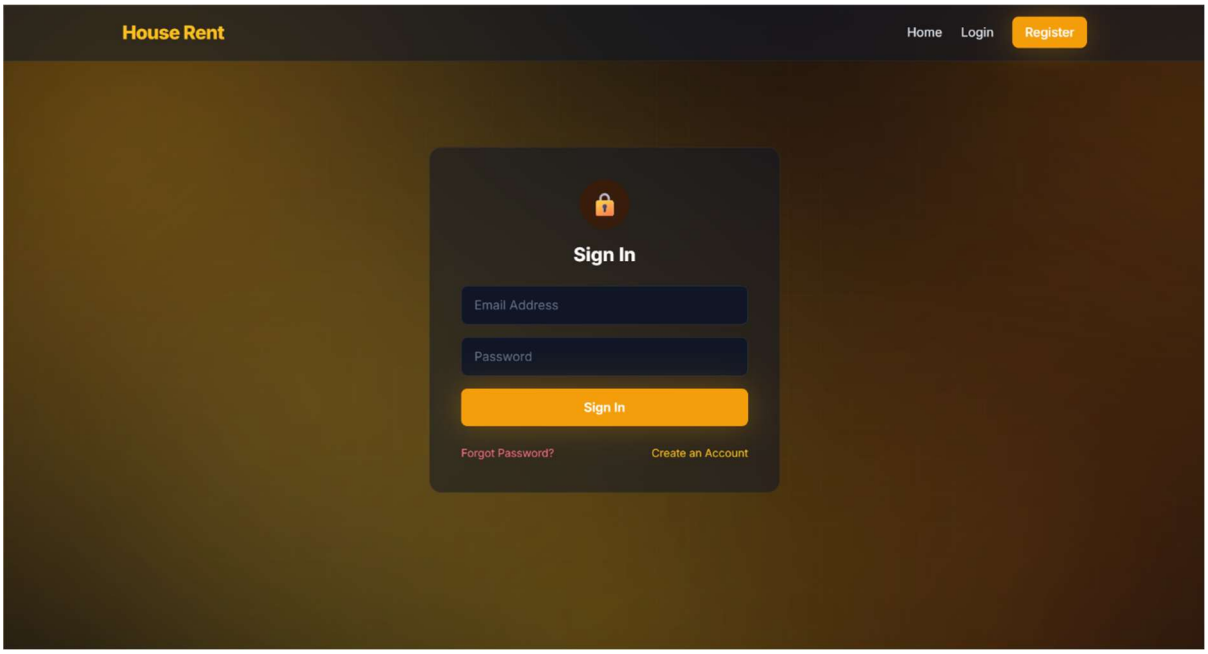

Figure 4: Login page

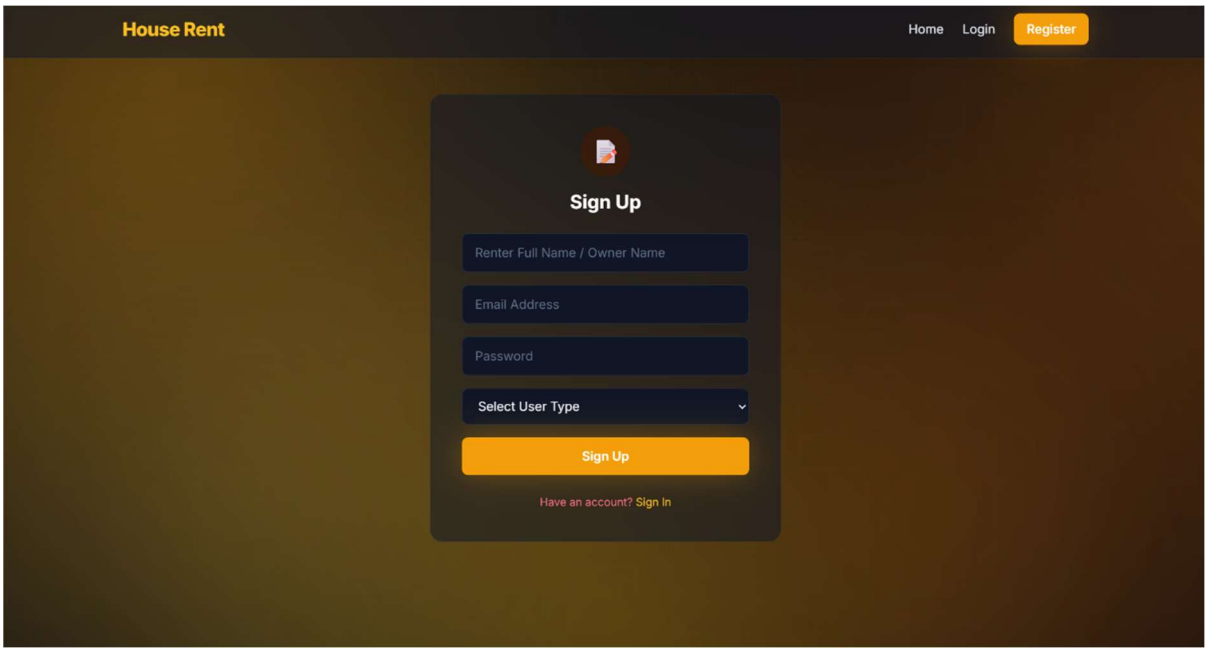

Figure 5: Register page

11.3 Owner Dashboard

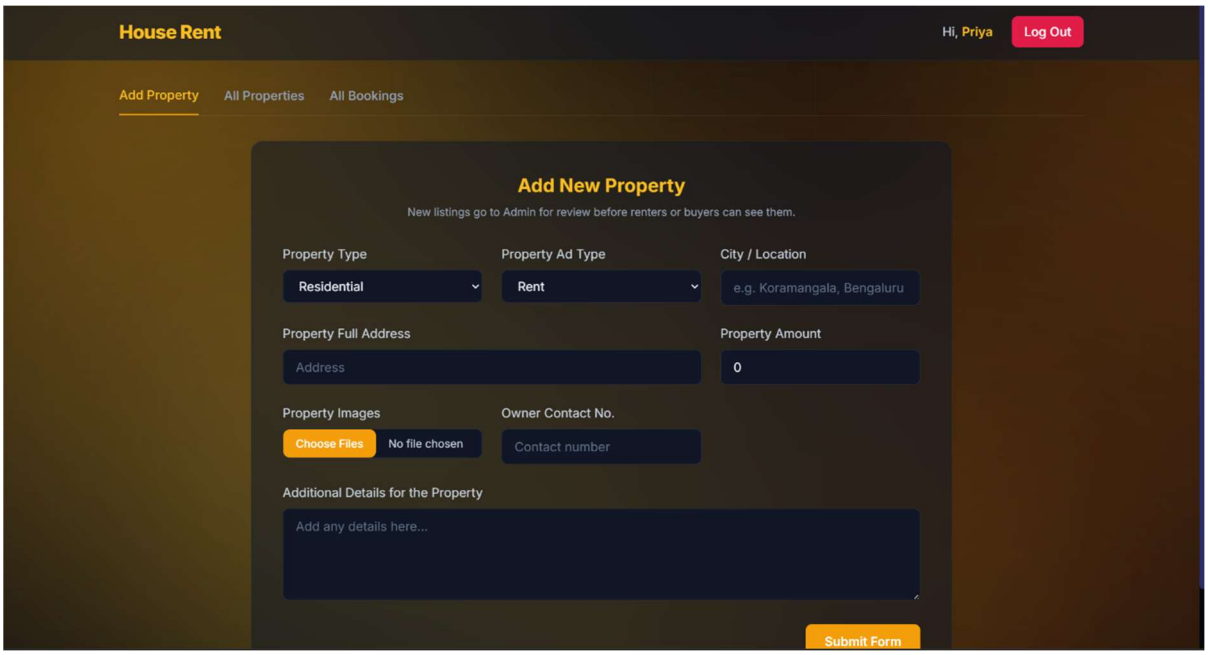

Figure 6: Owner — Add Property

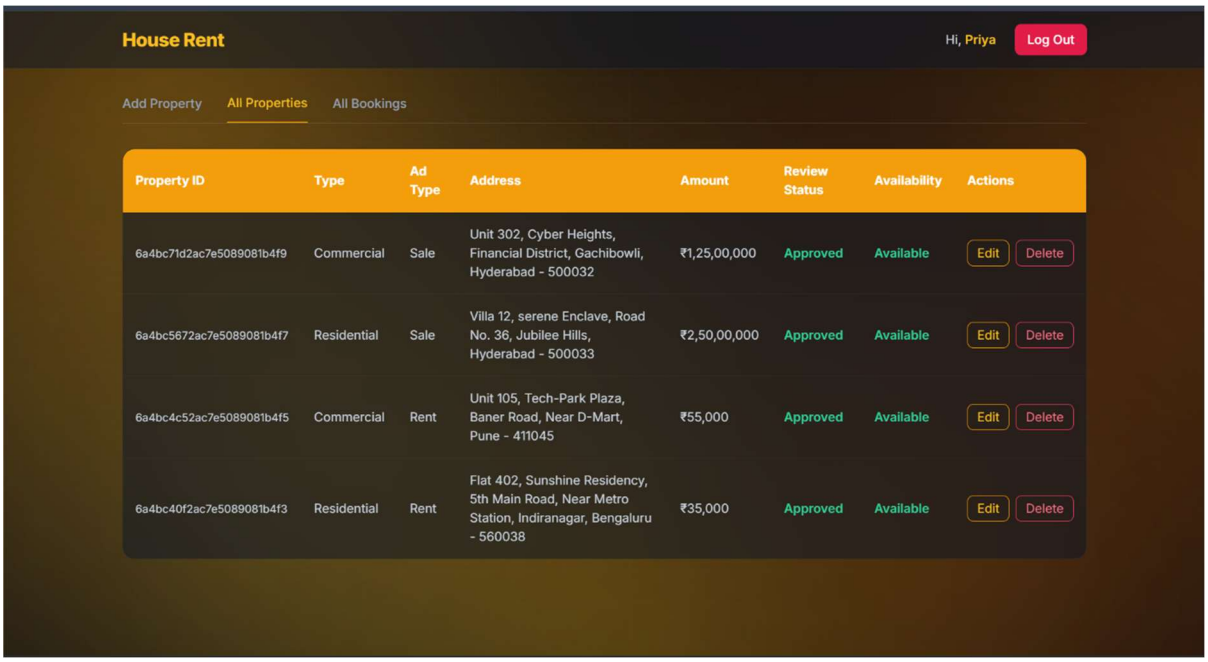

Figure 7: Owner — All Properties

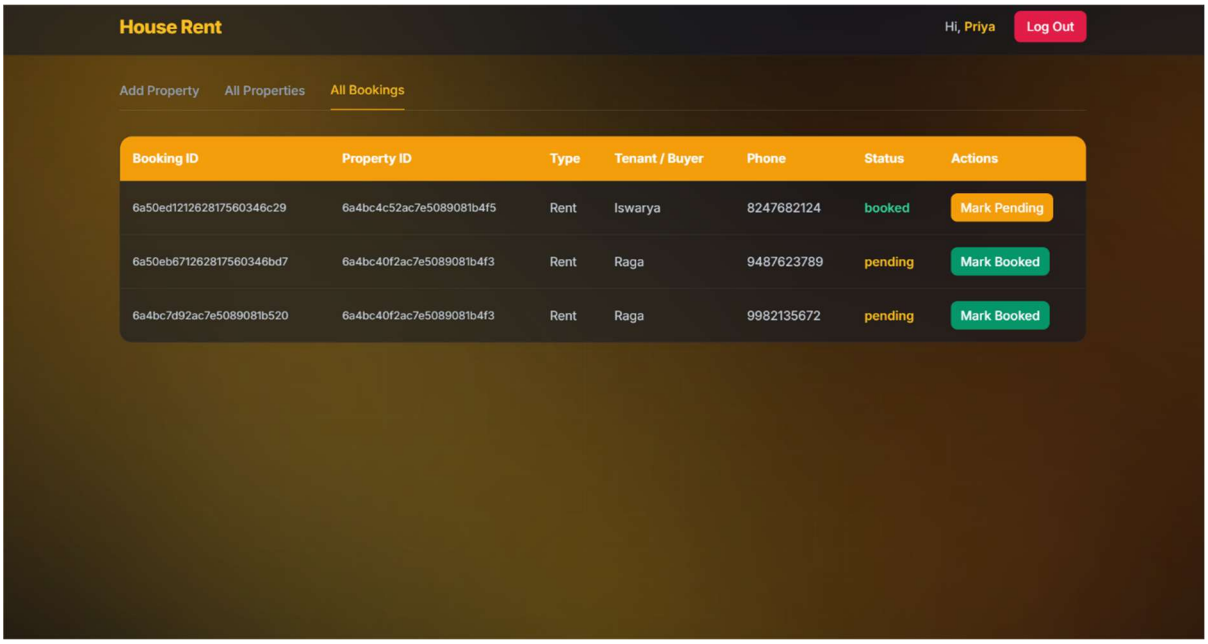

Figure 8: Owner — All Bookings

11.4 Admin Dashboard

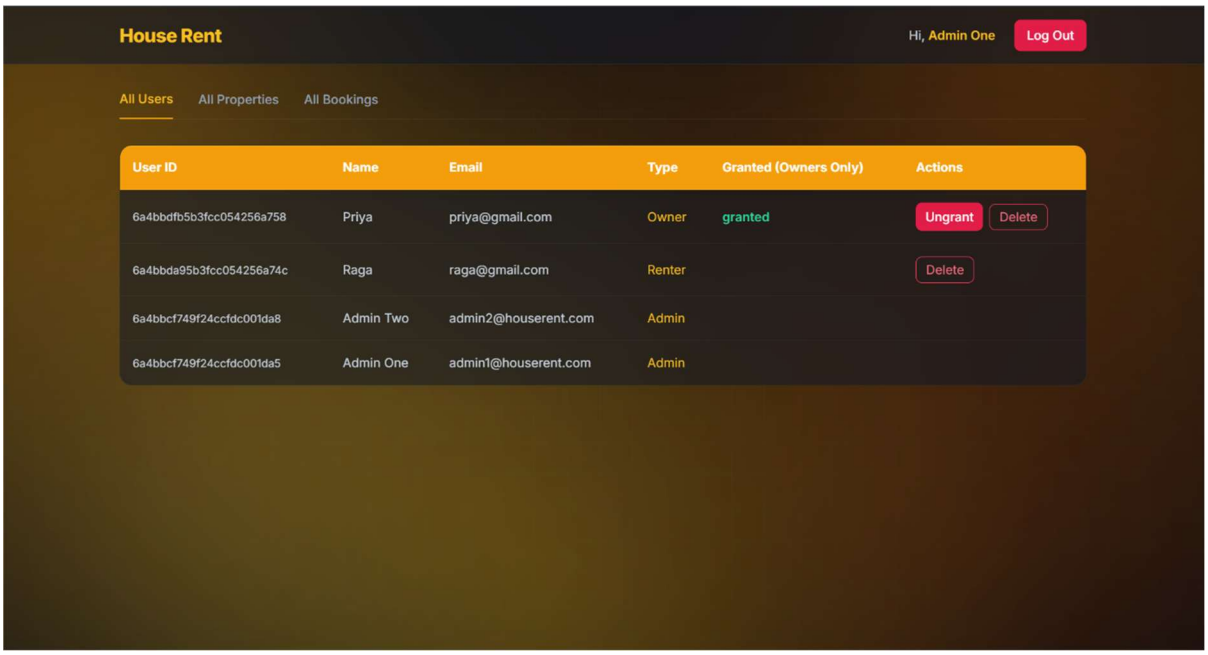

Figure 9: Admin — All Users

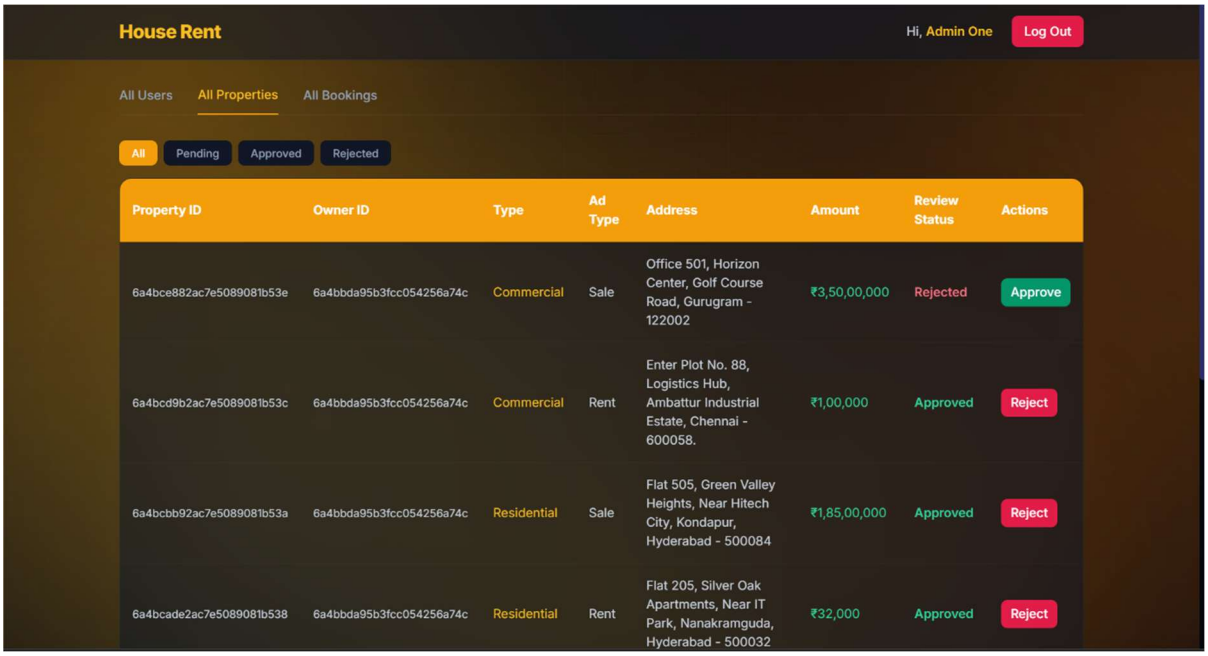

Figure 10: Admin — All Properties

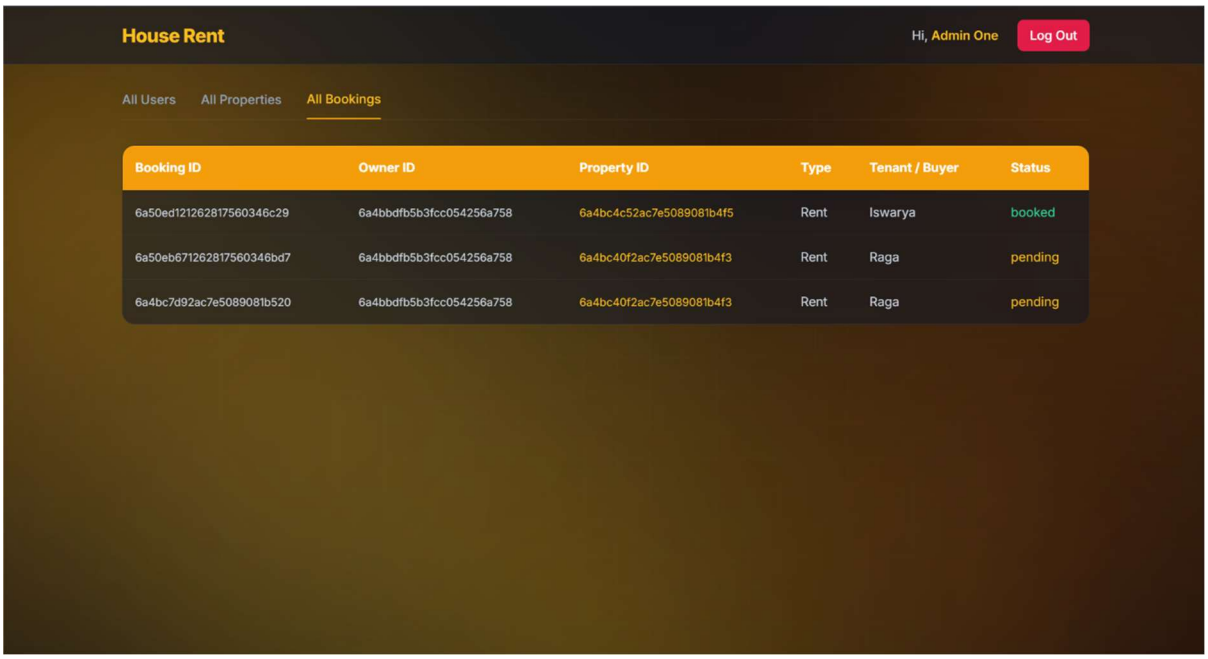

Figure 11: Admin — All Bookings

11.5 Renter Dashboard

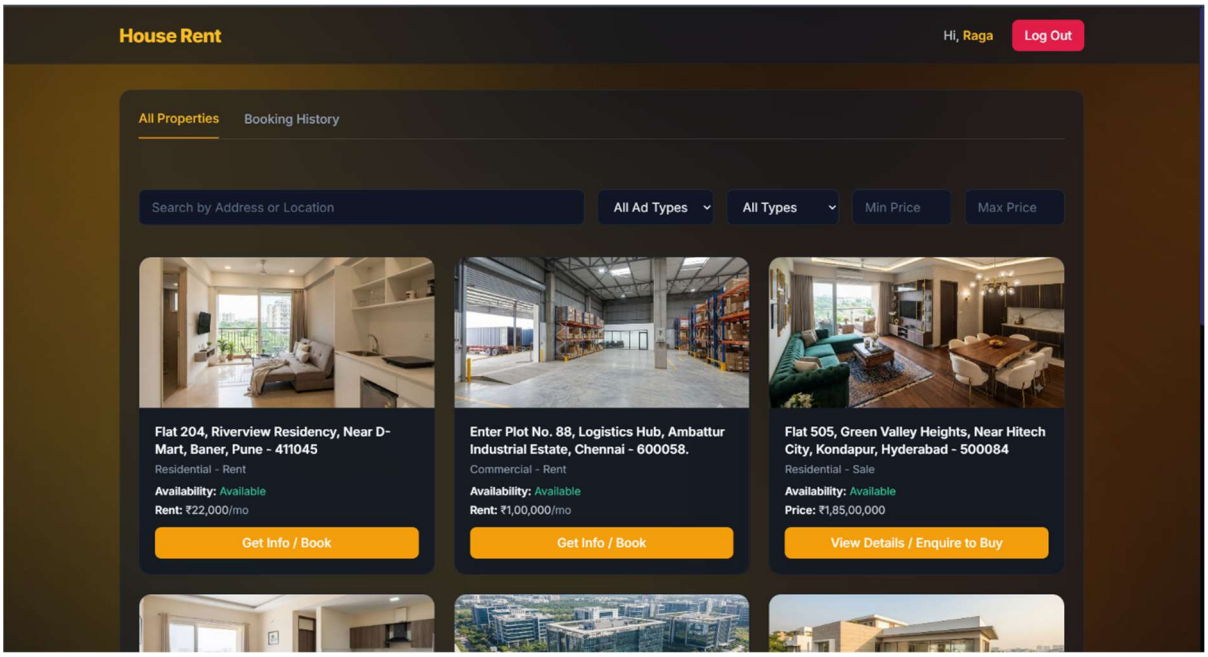

Figure 12: Renter — All Properties

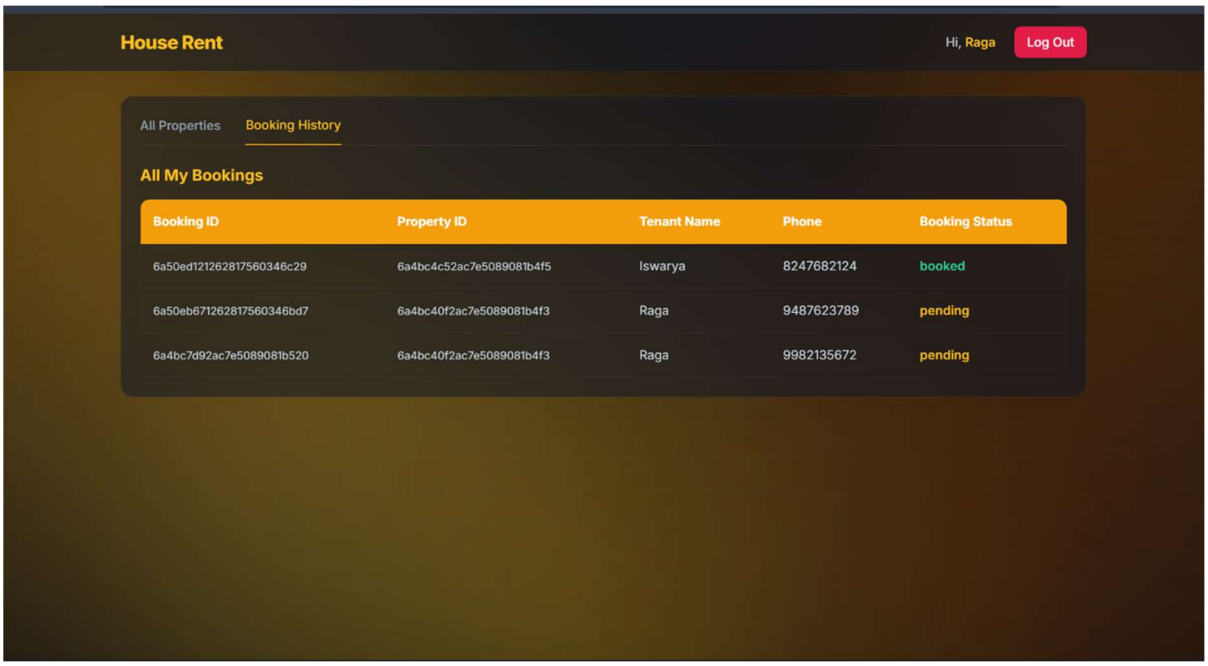

Figure 13: Renter — Booking History

12. Known Issues

- No automated test suite — the application currently relies on manual functional testing only; unit and integration tests are not yet part of the repository.
- No pagination on list views — the All Properties, All Users, and All Bookings tables load every record at once. This is fine at the current small scale but will need pagination or infinite scroll as data grows.
- Local file storage for uploads — property images are currently stored on the server's local disk rather than a cloud storage service or CDN, which will not scale across multiple server instances.
- No real-time updates — booking status changes, new listings, and approvals are only reflected after a page refresh; there is no live/push update mechanism.
- Environment configuration hygiene — .env files containing real database credentials must never be committed to version control; only a placeholder .env.example should be tracked in Git.